

SOULSIDE AI

TAKE-HOME ASSIGNMENT

Frontend Engineer — AI-Assisted Clinical Notes Workflow

Duration	Tech Stack	Deliverable	State Layer
5–7 Days	React + TypeScript	GitHub Repository	Candidate’s Choice

Overview

You are building the frontend for a clinical documentation platform where AI generates draft notes from recorded clinician-patient sessions, and licensed reviewers refine, approve, or reject them before the notes become part of the patient record. Your surface must feel calm and precise under load — clinicians open the app in the middle of back-to-back sessions and cannot afford to lose an edit, misfire a state transition, or be surprised by stale data.

This assignment is designed to evaluate how you reason about frontend system design: how you model a real domain, keep the UI honest with the server, hold consistency under concurrency, and scale a single-page application to hundreds of thousands of records without sacrificing responsiveness or accessibility. We are more interested in the architecture you choose and the trade-offs you can defend than in visual polish.

Domain Model

The product revolves around three domain concepts. Understanding the distinction between them — and how they relate — is critical to modelling the client state correctly.

Entity	Description	Key Attributes
Note	The reviewable unit of clinical documentation for one patient session. A Note is a container — its editable content lives in versions. The Note holds workflow state (status, assigned reviewer) and points to the current version.	id, patientId, sessionId, status, currentVersionId, assignedReviewerId, createdAt, updatedAt
NoteVersion	An immutable snapshot of the note’s content at a point in time. Every save produces a new version. Versions form a DAG: an amended note branches from an APPROVED ancestor. Diffs, blame, and rollback are all derived from the version graph.	versionId, noteId, revisionNumber, parentVersionId, content, authorId, authorRole, createdAt
ReviewEvent	An append-only audit record of every state transition and every user action on a note. The	eventId, noteId, versionId, fromStatus, toStatus, actorId, actorRole, reason, occurredAt

Entity	Description	Key Attributes
	review timeline you show to auditors is derived from this event log — not stored separately.	

i Relationship Summary

Note (1) → (*) NoteVersion: A single note accumulates versions across its lifetime. currentVersionId points to the head.

Note (1) → (*) ReviewEvent: Every action a user takes on a note produces an event; the review timeline is a projection over this log.

NoteVersion (1) → (0..1) NoteVersion: parentVersionId lets you reconstruct the version DAG, including amendment branches after APPROVED.

The server is authoritative for status transitions and for the head version. Your client must model both, keep them in sync, and gracefully surface disagreement.

Note Lifecycle

A note moves through a finite set of states. Your implementation must model this as a first-class state machine — not scattered if-checks — so that every UI affordance, every enabled action, and every allowed API call can be decided from (status, role, version) alone.

GENERATING → READY_FOR_REVIEW → IN_REVIEW → APPROVED → LOCKED

The full transition table:

From	To	Trigger	Guard
GENERATING	READY_FOR_REVIEW	generation.complete (auto)	—
GENERATING	FAILED	generation.error (auto)	—
FAILED	GENERATING	regenerate	role ∈ {CLINICIAN, ADMIN}
READY_FOR_REVIEW	IN_REVIEW	start_review	role = REVIEWER; assigns reviewerId
IN_REVIEW	READY_FOR_REVIEW	return	actor = assignedReviewer; releases lock
IN_REVIEW	APPROVED	approve	actor = assignedReviewer; MFA re-auth

From	To	Trigger	Guard
IN_REVIEW	REJECTED	reject	actor = assignedReviewer; reason required
REJECTED	READY_FOR_REVIEW	resubmit	role = CLINICIAN; creates new version
APPROVED	AMENDED	amend	within 24h grace; branches new version
APPROVED	LOCKED	grace_expired (auto, +24h)	—
AMENDED	IN_REVIEW	start_review	role = REVIEWER

⚠ Server is the Source of Truth

Every transition your UI initiates is optimistic — you assume success, update the client state, and reconcile with the server response. When the server rejects a transition (invalid state, stale version, insufficient permission, MFA required), your UI must roll back cleanly and surface the reason to the user.

Transitions can also arrive unsolicited over the real-time channel (another reviewer approved, a colleague started reviewing a note you were about to open). Your machine must accept server-driven transitions and, where they invalidate the user's current intent, interrupt gracefully — never silently drop them.

Sample API Payloads

Below are representative payload shapes for the endpoints your dummy backend must simulate. Field names are illustrative — you may adjust them, but keep the semantics.

GET /api/notes

```
{
  "cursor": {
    "next": "eyJvIjoxMDAsInMiOiJSRUFEWSJ9",
    "hasMore": true
  },
  "items": [
    {
      "id": "note_9f2a...",
      "patient": { "id": "pat_01H...", "displayName": "Riley A." },
      "status": "READY_FOR_REVIEW",
      "currentVersion": { "id": "ver_...", "revision": 3 },
      "assignedReviewer": null,
      "createdAt": "2025-11-04T14:22:10Z",
      "updatedAt": "2025-11-04T14:41:02Z"
    }
  ]
}
```

```
],
  "meta": { "total": 132491, "returned": 50, "generatedAt": "2025-11-04T14:41:03Z" }
}
```

GET /api/notes/:id

```
{
  "id": "note_9f2a...",
  "patient": { "id": "pat_01H...", "displayName": "Riley A." },
  "status": "IN_REVIEW",
  "assignedReviewer": { "id": "usr_...", "displayName": "Dr. Chen", "role": "REVIEWER" },
  "currentVersion": {
    "id": "ver_7bd...",
    "revision": 5,
    "parentVersionId": "ver_6ac...",
    "content": { "sections": { "S": "...", "O": "...", "A": "...", "P": "..." } },
    "authoredBy": { "id": "usr_...", "role": "CLINICIAN" },
    "createdAt": "2025-11-04T14:41:02Z"
  },
  "versions": [
    { "id": "ver_7bd...", "revision": 5, "parentVersionId": "ver_6ac...", "authoredBy":
{...} }
  ],
  "review": {
    "events": [
      { "id": "evt_1", "fromStatus": "READY_FOR_REVIEW",
        "toStatus": "IN_REVIEW", "actorId": "usr_...", "occurredAt": "..." }
    ]
  }
}
```

POST /api/notes/:id/versions

```
// Request - every save carries the base version the client edited from.
// The server rejects the save (409) if baseVersionId ≠ current head.
{
  "baseVersionId": "ver_7bd...",
  "content": { "sections": { "S": "...", "O": "...", "A": "...", "P": "..." } },
  "clientMutationId": "mut_01HZK..." // idempotency key
}

// Success response
{ "version": { "id": "ver_8ce...", "revision": 6, "parentVersionId": "ver_7bd..." } }

// Conflict response (409)
{
  "error": "version_conflict",
  "current": { "id": "ver_9df...", "revision": 7, "authoredBy": {...} },
  "commonAncestor": { "id": "ver_6ac...", "revision": 4 }
}
```

Real-Time Channel (WebSocket / SSE)

```
// Server → client. One channel per open note plus a workspace-wide feed.
{ "type": "note.status_changed",
  "noteId": "note_...", "fromStatus": "READY_FOR_REVIEW",
  "toStatus": "IN_REVIEW", "actor": { "id": "usr_...", "displayName": "Dr. Chen" },
  "at": "2025-11-04T14:41:02Z", "eventId": "evt_..." }

{ "type": "note.version_added",
  "noteId": "note_...", "version": { "id": "ver_8ce...", "revision": 6 } }

{ "type": "note.presence",
  "noteId": "note_...", "viewers": [ { "id": "usr_a", "role": "REVIEWER" } ] }
```

Payload Notes

- Pagination is cursor-based; do not assume offset semantics or a stable ordering across pages.
- `clientMutationId` must be treated as an idempotency key. Duplicate deliveries (retries, offline replay) must not create duplicate versions.
- On `version_conflict`, the server returns the current head plus the last common ancestor. Your UI must resolve this — do not overwrite silently and do not force the user to reload.
- Real-time events may arrive before the acknowledgment of the mutation that caused them. Reconcile by `eventId`; do not rely on ordering.
- Real-time events may also be dropped or duplicated. Design for at-least-once delivery.

Functional Requirements

Notes List View

- A cursor-paginated, virtualized list that stays responsive against a dataset of 100,000+ notes without loading them all into memory.
- Filter panel: multi-select status, assigned reviewer, patient, date range. Filter state must be URL-persisted and deep-linkable.
- Server-side search across patient name and note content with a debounced query. Empty and no-results states must be distinct.
- Sortable columns with a stable secondary sort; sort state also URL-persisted.
- Bulk actions (assign reviewer, request regeneration) on selected rows. Selection must survive pagination and filter changes for as long as the row is in view.
- Skeleton loaders and optimistic row updates so the list never jumps or blinks when a status changes.

Note Detail View

- Structured editor for SOAP sections (Subjective, Objective, Assessment, Plan). Each section is independently dirty-tracked.
- Version history sidebar with a diff view between any two versions. Word-level highlighting is expected; character-level is a plus.

- Status-driven action bar. Every action button is derived from the state machine — disabled actions must show the reason ("You are not the assigned reviewer", "Note is LOCKED after 24h grace").
- Read-only presentation when the note is in LOCKED status, with a clear affordance to start an amendment.
- Presence indicator: show who else is currently viewing or reviewing this note, live.

State Machine Engine

- Model the lifecycle as a first-class, pure, unit-testable module. No component should hard-code an if-check against a status string.
- Every user-initiated action must be validated against the machine before an API call is issued, and rolled back if the server rejects it.
- Server-pushed transitions (from the real-time channel) must run through the same machine so client-side invariants are preserved.
- Emit a local ReviewEvent immediately on optimistic transition; reconcile with the server-assigned eventId on ack.

Autosave & Version Conflict Handling

- Autosave the working draft on a debounced interval while the editor is dirty.
- Coalesce in-flight saves — never allow two concurrent POSTs for the same note. If a save is in flight and the content changes, queue exactly one follow-up save.
- Every save must carry baseVersionId. On a 409 version_conflict, present a three-way diff (your changes • server head • common ancestor) and let the user resolve — do not overwrite, do not force a reload.
- Every save must carry a clientMutationId so retries are safely idempotent.

Offline & Resumability

- The app must remain usable for at least 30 minutes without a network. Reads served from cache, writes queued locally.
- The write queue must survive a full page reload — persist to IndexedDB and replay on reconnect.
- On reconnect, replay writes in order, honouring baseVersionId; surface conflicts through the same three-way merge UI.
- Show an unambiguous, non-modal connectivity status. Users must never be uncertain whether their last edit was saved.

Real-Time Updates

- Subscribe to the real-time channel for the notes currently on screen (list rows in view + open detail). Unsubscribe when they leave the viewport.
- Merge server pushes with local optimistic state. If the server-pushed version supersedes an in-flight optimistic edit, the resolution UI is the same three-way merge.
- Reconnect with exponential backoff and jitter. On reconnect, request a replay of missed events since the last known cursor — do not assume the socket dropped nothing.

Telemetry

- Provide a `track(name, properties, {important?})` abstraction. No component should call the telemetry endpoint directly.
- Batch events client-side. Flush on a size threshold, a time threshold, and session boundaries (route change, tab hidden).
- Retry failed batches with backoff. After N failed attempts, park the batch in IndexedDB for a later attempt.
- Do not lose events on unload. Use `sendBeacon` (or `fetch keepalive`) for the final flush.
- Run a PII redaction pass before every send. Free-text note content must never appear in telemetry.

Authorization

- Roles: CLINICIAN, REVIEWER, ADMIN, READONLY_AUDITOR. Every UI affordance is a function of (role, status, ownership).
- Route-level guards, component-level guards, and action-level guards. Missing permission is not the same as missing data — messaging must be distinct.
- Client-side authorization checks are UX only. The server is authoritative; assume a hostile client and design your abstraction so a rogue button click cannot silently bypass a guard.

Dummy Backend

- Simulate the REST endpoints with realistic latency (100–800 ms) and 5% failure injection so timeout, retry, and offline logic can be exercised locally.
- Ship a mock real-time channel (WebSocket or SSE) that emits status changes, version additions, and presence on a schedule.
- Seed the store deterministically so simulation runs are reproducible.

Simulation Script

We have provided a Node script that drives your dummy backend through a realistic multi-user workflow: it seeds a dataset, simulates concurrent reviewers, injects network flakiness, and exercises version conflicts. Use it to validate your implementation.

```
// simulate_workflow.ts - Happy-path + light concurrency simulation
// Usage: npm run tsx simulate_workflow.ts [BASE_URL]
//
// This script simulates a realistic reviewer workday:
// 1. Seed the store with 5,000 notes across mixed statuses
// 2. Spawn 3 reviewer "actors" who each pick READY_FOR_REVIEW notes,
//    start a review, apply 1-3 edits, then approve or reject
// 3. Inject 5% API latency spikes and 3% 500-error responses
// 4. Push a version_conflict for 2% of saves
// 5. Emit real-time status_changed events on a 250ms cadence

const BASE = process.argv[2] ?? "http://localhost:5173";
```

```
const seed = async () => { /* POST /api/dev/seed { count: 5000 } */ };

const reviewerLoop = async (reviewerId: string) => {
  for (let i = 0; i < 20; i++) {
    const note = await pickReadyNote();
    await post(`/api/notes/${note.id}/transitions`,
      { to: "IN_REVIEW", actorId: reviewerId });

    for (let e = 0; e < 1 + rand(3); e++) {
      const head = await get(`/api/notes/${note.id}`);
      await post(`/api/notes/${note.id}/versions`, {
        baseVersionId: head.currentVersion.id,
        content: mutateContent(head.currentVersion.content),
        clientMutationId: uuid(),
      });
    }

    const outcome = Math.random() < 0.7 ? "APPROVED" : "REJECTED";
    await post(`/api/notes/${note.id}/transitions`,
      { to: outcome, actorId: reviewerId, reason: outcome === "REJECTED" ?
"missing plan" : undefined });
  }
};

await seed();
await Promise.all(["dr_a", "dr_b", "dr_c"].map(reviewerLoop));
console.log("=== Simulation complete ===");
console.log("Verify: telemetry batches sent, no lost writes, no unhandled conflicts");
```

⚠ Build Your Own Test Scenarios

The script above exercises the happy path plus light concurrency. We expect you to construct additional scenarios to prove your architecture. Think about:

- Two reviewers editing the same note in overlapping windows — one wins, the other's UI must resolve the conflict without losing their work.
- A network drop mid-save while the autosave queue holds three pending mutations, then reconnect 20 minutes later.
- A real-time note.status_changed arriving before the acknowledgment of the mutation that caused it.
- A REJECTED note resubmitted after the base version has been superseded by an admin edit.
- Reviewing 500 notes back-to-back in one session — memory, listeners, and subscriptions should not leak.

How your application handles these cases, and how you document your decisions, is a substantial part of the evaluation.

Required Design Decisions

Your README must explicitly address the following. We expect written reasoning, not just a checklist of libraries.

- **State Topology** — Where state lives

- **State Machine** — How the note lifecycle is modelled and how invalid transitions are prevented
- **Optimistic Updates** — How optimistic updates are applied and rolled back on failure
- **Concurrency** — How version conflicts are surfaced and resolved without data loss
- **Offline** — How the write queue survives reloads and how replay handles conflicts
- **Real-Time** — How the real-time channel is reconciled with local optimistic state
- **Telemetry** — How events are batched, retried, and made resilient to unload
- **Scale** — How the list, detail, and history views stay responsive at 100k+ notes
- **Testing** — What you tested at unit, integration, and end-to-end level, and why
- **Accessibility** — Keyboard support, screen-reader semantics, and your WCAG 2.2 AA posture

Evaluation Criteria

We will evaluate your submission across the following dimensions. There are no trick questions — we want to understand how you think, not just what you can produce.

Criterion	What We Look For
Frontend Architecture	Clean layering, one-way dependency direction, module boundaries that would let the team swap the data layer, the transport, or the UI framework without a rewrite. Deliberate choices about where state lives.
State Modelling & Invariants	A first-class, testable state machine. Transitions validated in one place. UI affordances derived from the machine, not from ad-hoc conditionals. Server-driven transitions handled through the same path.
Concurrency & Consistency	Optimistic updates that roll back cleanly. Version conflicts resolved without data loss. At-least-once real-time delivery handled explicitly. Idempotent mutations.
Performance & Scale	Responsive rendering at 100k+ notes. Virtualization, memoization, and query design that reflect a real understanding of the render and network cost model. No unbounded subscriptions or listener leaks.
Async & Effect Discipline	Race conditions accounted for. Cancellation on unmount. Backpressure on telemetry and the write queue. Fetches keyed correctly so stale responses don't clobber current state.
Accessibility	Keyboard-navigable end to end. Screen-reader semantics for the editor, status changes, and conflict resolution. A stated WCAG 2.2 AA posture and any documented gaps.
Testing Strategy	Unit tests for the state machine and the pure logic. Integration tests for effectful modules (autosave, real-time reconciliation, offline replay). A smoke path for the critical user flows. A short note explaining what you chose to test and what you chose not to.

Criterion	What We Look For
Design Considerations	Error handling, logging, authorization posture, PII redaction, and documented trade-offs. A short design document is welcomed.

Deliverables

- A GitHub repository (public or private, with access shared) containing the complete source code.
- A README.md that includes: build/run instructions, an architecture overview, the design decisions and trade-offs you made, and any assumptions.
- The state-machine specification, either as source of truth (a code module) or as a diagram plus a link to the code that implements it.
- Your own test scenarios (scripts, unit/integration tests, or a Playwright/Cypress suite) beyond the provided simulation script.
- (Optional) A short architecture diagram showing state topology, effect flow, and how the real-time channel is reconciled with local state.



Important Notes

- You do NOT need to deploy the application. A locally runnable project is sufficient.
- You do NOT need a production real-time broker; a mock WebSocket/SSE server is acceptable.
- You do NOT need production-grade UI polish. Focus on architecture, correctness, and the invariants that hold under load.
- Focus on quality over quantity. A small, coherent, well-argued solution is preferred over a broad, shallow one.

Bonus (Not Required)

The following are strictly optional, but will be positively noted if present:

- Real-time collaborative editing via a CRDT (Yjs, Automerge) with cursor presence.
- Word-level or character-level diff view between any two versions in the version graph.
- A plugin architecture for note-type-specific editors (SOAP, DAP, BIRP) so a new format is a plugin, not a fork.
- Feature-flag / experimentation harness with per-user rollout.
- Structured logging with correlation IDs linking a user action to its network request, telemetry event, and real-time echo.
- Web Workers for diffing, large-document parsing, or telemetry serialization.
- Progressive Web App: installable, offline-first, background sync of the write queue.
- Micro-frontend / module federation exploration with a written rationale.

Timeline & Submission

Please complete and submit your solution within 5–7 days of receiving this assignment. If you need additional time, please let us know — we are happy to accommodate reasonable requests.

When you are ready, share the repository link (or invite us as collaborators if the repository is private) along with any additional notes or context you would like us to consider.



Questions?

If anything is unclear or you have questions about the scope, requirements, or expectations, please don't hesitate to reach out. We'd rather you ask than make assumptions that take you in the wrong direction.

Good luck! The Soulside AI team looks forward to reviewing your work.